赛后总结

这里是嘉兴一中 2026 年的赛后总结帖。

感谢各位对本场比赛的支持！

本场比赛共有 9 人参与，分数非 0 人数共 3 人，AK 人数共 0 人。

各题具体情况如下：

题目	通过人数	通过率 & 得分率	预估难度	预期
A	1	$33.3\%, 75\%$	普及-，橙	符合预期
B	0	$0\%, 50\%$	普及，黄	低于预期
C	0	$0\%, 25\%$	提高，绿	远低于预期
D	0	$0\%, 75\%$	提高+，蓝	符合预期

- 注：通过率 = “该题通过人数” 除以 “有提交的选手数量”；得分率 = “该题得分人数” 除以 “选手总数”。

A

> Tips1：如何转化题意？
> 目标是颜色相间，那么只有两种情况：
> `0101010...101` 或者 `1010101...010`。

> Tips2
> 只要让所有的 0 符合目标位置，那么 1 自然也就在符合的位置，反之亦然。

> Tips3：如何匹配？
> 现在问题转化为：当前 0 的位置是 $p_1, p_2, \ldots, p_n$，目标位置是 $0, 2, 4, \ldots, 2n-2$ 或 $1, 3, 5, \ldots, 2n-1$，如何移动使得> 达到目标的步数最少

解法：因为 $p_1 < p_2 < \ldots < p_n$，所以我们一定是让 p_1 到 0，p_2 到 2，等等这样贪心的匹配。

使用反证法即可证明。

于是枚举两种目标位置即可。那么本题做完了。注意答案与 n^2 同阶，**要开 long long**。

```cpp
#include <bits/stdc++.h>
using namespace std;
int n;
string s;
inline void sol() {
    cin >> n >> s;
    vector<int> pos;

```

```
    for(int i = 0; i < (n << 1); ++i)
        if(s[i] == '0')
            pos.push_back(i);
    long long ans1 = 0, ans2 = 0;
    for(int i = 0; i < n; ++i)
        ans1 += abs(pos[i] - (i << 1));
    for(int i = 0; i < n; ++i)
        ans2 += abs(pos[i] - ((i << 1) + 1));
    cout << min(ans1, ans2) << '\n';
}
signed main() {
    freopen("sort.in", "r", stdin), freopen("sort.out", "w", stdout);
    cin.tie(0), cout.tie(0), ios::sync_with_stdio(0);
    int T; cin >> T;
    while(T--) sol();
    return 0;
}
```

B

使用优先队列模拟即可。具体看代码实现。

```
#include<bits/stdc++.h>
typedef long long ll;
using namespace std;
typedef pair<ll, int> PII;
priority_queue<PII, vector<PII>, greater<PII>> q;
int n, k;
signed main() {
    freopen("dine.in", "r", stdin), freopen("dine.out", "w", stdout);
    cin.tie(0), cout.tie(0); ios::sync_with_stdio(0);
    cin >> n >> k;
    int cur = 0;
    ll time = 0;
    for(int i = 0; i < n; ++i) {
        int a, b, c;
        cin >> a >> b >> c;
        while(cur + c > k) {
            PII tmp = q.top(); q.pop();
            time = tmp.first, cur -= tmp.second;
        }
        ll ans = max(time, (ll)a);
        cout << ans << '\n';
```

```
        q.emplace(ans + b, c);
        cur += c;
    }
    return 0;
}
```

C

观察到特殊性质 A，启发我们往 DP 的角度思考。

又注意到数据范围 $n, m \le 10^3$，考虑 $\mathcal{O}(nm\omega)$ 的 DP。

n 个数，限定 m 次操作，不难想到背包状 DP。设 $f_{i,j}$ 表示考虑前 i 个数用了 j 次操作的最大和。有：

$$f_{i,j} = \max_{k=0}^{31}\{f_{i-1,j-k} + a_i \overline{\ll} k\}$$

其中 $\overline{\ll}$ 表示循环左移，在枚举 k 的过程中维护。具体地说，x 循环左移一位，就是其正常左移一位，再将其原本最高位上的数加到最低位上。写成代码就是 `x << 1 | (x >> 31 & 1)`。

背包 DP 可以使用滚动数组优化，即将 j 倒序枚举。

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int N = 1010;
int n, m;
unsigned a[N];
ll f[N], ans;
signed main() {
    freopen("recall.in", "r", stdin), freopen("recall.out", "w", stdout);
    cin.tie(0), cout.tie(0); ios::sync_with_stdio(0);
    cin >> n >> m;
    for(int i = 1; i <= n; ++i) cin >> a[i];
    for(int i = 1; i <= n; ++i)
        for(int j = m; ~j; --j) {
            unsigned x = a[i];
            for (int k = 0, u = min(j, 31); k <= u; ++k) {
                f[j] = max(f[j], f[j - k] + x);
                x = x << 1 | (x >> 31 & 1);
            }
        }
    for(int i = 0; i <= m; ++i) ans = max(ans, f[i]);

```

```
    cout << ans;
}
```

D

绝世好题。原：P14665。

Part I

先考虑 $n, m \le 5 \times 10^3$ 怎么做。

对任意一个区间执行减 1 操作，等价于对该区间之外的其余部分执行加 1 操作，两种操作对序列极差的影响完全相同。

故我们可以把序列 a 看作一个环，就可以将题目转化为只有区间加 1 这一种操作，只需确定每次操作的区间。

每次修改的区间**必定包含最小值，且不包含最大值**。显然，选取更长的这类区间是更优的。

> Q：为什么更优？
> A：选取最长的这类区间，由于不含最大值，会使更多的数更接近最大值而不会改变最大值。

所以，每次对最长的包含最小值且不包含最大值的区间执行加 1 操作。

每次操作，使用 $\mathcal{O}(n)$ 的时间复杂度来寻找目标区间并将其增加 1，总时间复杂度 $\mathcal{O}(nm)$。

```
#include <bits/stdc++.h>
using namespace std;
int n, m, a[5010];
inline int id(int i) {  return (i - 1) % n + 1; }
inline void sol() {
    cin >> n >> m;
    for(int i = 1; i <= n; ++i) cin >> a[i];
    for(int i = 1; i <= m; ++i) {
        int mx = 0, mn = 1e6;
        for(int i = 1; i <= n; ++i)
            mx = max(mx, a[i]), mn = min(mn, a[i]);
        int l = 0, len = -1e9, pl, pr;
        bool f = 0; // f 表示区间内是否有最小值
        for(int i = 1; i <= (n << 1); ++i) {
            if(a[id(i - 1)] == mx) l = i, f = 0;
            if(a[id(i)] == mn && l > 0) f = 1;
            if(a[id(i)] == mx && f && i - l > len)
                pl = l, pr = i - 1, len = i - l;
        }
        for(int i = pl; i <= pr; ++i)
            ++a[id(i)];

```

```
    }
    int mx = 0, mn = 1e6;
    for(int i = 1; i <= n; ++i)
        mx = max(mx, a[i]), mn = min(mn, a[i]);
    cout << mx - mn << '\n';
}
signed main() {
    freopen("delta.in", "r", stdin), freopen("delta.out", "w", stdout);
    cin.tie(0), cout.tie(0) -> sync_with_stdio(0);
    int T;
    cin >> T;
    while(T--) sol();
    return 0;
}
```

Part II

简述一下 $n, m \le 10^5$ 的做法，对正解有启发作用。

观察到极差这个属性具有单调性：如果我们能构造出极差为 k 的序列，那么一定能构造出极差为 $k+1$ 的序列。故我们**二分极差 k**。

于是题目便转化为了一个可行性问题：判断是否能使序列极差 $\le k$，即二分的 `check` 函数。

要使极差为 k，换言之序列 a 中所有数的值应在区间 $[v, v+k]$ 内。令 $f(v)$ 表示将所有数的值调整到区间 $[v, v+k]$ 所需的最少操作次数，不难得到 $f(v)$ 是单峰函数。可以用三分优化，因此 `check` 函数时间复杂度 $\mathcal{O}(n \log n)$，总时间复杂度 $\mathcal{O}(n \log^2 n)$。

Part III

正解。瓶颈在于 `check` 函数的复杂度，怎么办呢？对于这类区间加减问题，通常考虑**差分**。

然后你会发现极差其实就是差分数组的最大子段和或最小子段和。我们把这个叫做绝对值最大子段和。

因为只和差分数组有关，于是接下来我们只需要考虑差分数组。

如果当前的这个绝对值最大子段和超出正在检验的答案，那么就要进行调整。

这个调整对于正负两个计数器都会有效果，所以累加答案的同时也要对这两个同时进行修改。

我们把需要进行的增加操作与减去操作分开来存，因为一次操作事实上可以同时完成一个增加和一个减去，于是只需要这二者操作次数的最大值即可。

显然可以把操作点放到序列外面从而把两个操作位置变成零个甚至一个，因此多出来的操作机会可以不管。

于是做完了。时间复杂度 $\mathcal{O}(n \log n)$。

PS：本题做法非常多，欢迎读者进一步思考。

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int N = 2e5 + 10;
int n, m, a[N], b[N];
inline bool chk(int x) {
    ll mx = 0, mn = 0, mxuse = 0, mnuse = 0;
    for(int i = 2; i <= n; ++i) {
        mx = max(mx + b[i], (ll)b[i]), mn = min(mn + b[i], (ll)b[i]);
        if(mx > x) mxuse += mx - x, mn -= mx - x, mx = x;
        if(-mn > x) mnuse += -mn - x, mx += -mn - x, mn = -x;
    }
    return max(mxuse, mnuse) <= m;
}
inline void sol() {
    cin >> n >> m;
    int pl = 1e6, pr = 0, ans;
    for(int i = 1; i <= n; ++i) {
        cin >> a[i];
        b[i] = a[i] - a[i - 1];
        pl = min(pl, a[i]), pr = max(pr, a[i]);
    }
    ans = (pr -= pl);
    pl = 0;
    while(pl <= pr) {
        int mid = (pl + pr) >> 1;
        if(chk(mid)) pr = (ans = mid) - 1;
        else pl = mid + 1;
    }
    cout << ans << '\n';
}
signed main() {
    freopen("delta.in", "r", stdin), freopen("delta.out", "w", stdout);
    cin.tie(0), cout.tie(0); ios::sync_with_stdio(0);
    int T;
    cin >> T;
    while(T--) sol();
    return 0;
}
```